Application Flow Document

Product: ExamGuard — Smart Exam Allocation & Identity Verification System **Document Version:** 1.0 **Date:** August 14, 2026

1. End-to-End Lifecycle Overview

```
[1] Setup Phase          [2] Pre-Exam Phase        [3] Reveal Phase         [4] Verification Phase      [5] Post-Exam Phase
--------------------     --------------------      --------------------     -------------------------  -------------------
Admin uploads roster  →  Student views timetable →  T-30: hall/seat    →   Invigilator scans barcode → Admin reviews logs,
creates halls/exams      and locked pass             auto-unlocks on         checks identity, verifies   exports attendance,
maps students to                                     student portal          or flags student            resolves anomalies
hall + seat
```

2. Actor Journeys

2.1 Administrator Journey

1. **Login** to Admin Web Console with institutional credentials.
2. **Upload roster** (CSV/Excel): roll number, name, email, department, photo reference. System validates and returns a per-row error report if applicable; admin corrects and re-uploads until clean.
3. **Create halls**: building name, room number, floor, capacity — done once per venue, reused across exam cycles.
4. **Create exam**: course code, title, date, start time, end time.
5. **Map students**: select a roster segment (e.g., a department or course cohort), choose one or more halls, and either auto-allocate seats by capacity or manually assign via a seat-map grid. System generates a unique signed `barcode_token` per mapping.
6. **Configure reveal threshold** for the exam (default T-30 minutes), with a live preview of the exact unlock time.
7. **Monitor exam day**: open the Live Attendance Dashboard; watch hall-by-hall verified/pending/absent/wrong-hall counts update in real time as invigilators scan.
8. **Post-exam**: review flagged/wrong-hall/absent records, export the audit log, close out the exam session.

2.2 Student Journey

1. **Login** to the Student Web Portal using roll number and password (forced reset on first login).
2. **View timetable**: all mapped exams listed chronologically with date/time and status.
3. **Open exam pass**: see course details, personal photo (self-verification), and a dynamic, auto-refreshing barcode.
4. **Hall/seat locked state**: prior to T-30 minutes, the hall/seat panel shows "Locked" with a live countdown.
5. **Reveal**: at exactly T-30 minutes before start, the panel unlocks automatically (via a pushed WebSocket event, no refresh needed) to show building, room number, and seat number.
6. **Proceed to hall**: student presents the barcode (on-screen or printed) at the hall entrance.
7. **Verification confirmation**: after the invigilator scans and verifies, the student's portal reflects a "Checked In" status if they check back.

2.3 Invigilator Journey

1. **Login** to the Expo Scanner App with invigilator credentials.
2. **Select assigned hall/exam** (or it is pre-loaded based on roster assignment).
3. **Pre-sync roster**: before the exam window, the app downloads the full hall roster and student photo thumbnails into local offline storage; a readiness indicator confirms caching is complete.
4. **Scan at the door**: point the camera at each student's barcode as they arrive.
5. **Review verification card**: candidate photo, name, roll number, course, assigned hall, and seat appear instantly.
6. **Visual decision cue**: - Green flash → correct hall and seat → tap "Verify" (or auto-confirm) → student proceeds in. - Red flash → wrong hall → app displays the correct hall/room, student is redirected. - Amber/duplicate flash → token already used → invigilator investigates (possible shared screenshot) or defers to manual search.
7. **Fallback path**: if a barcode is damaged or a phone is dead, invigilator uses Manual Roll Number Search to pull up the same verification card.
8. **Offline operation**: if connectivity drops, scanning continues uninterrupted against the local cache; each verification is queued locally.
9. **Sync**: when connectivity returns (automatically detected), queued verification events sync to the server in the background; the sync status bar reflects pending/synced counts throughout.
10. **Session close**: at the end of the exam window, invigilator reviews a summary (verified/absent/flagged) and submits it to the admin dashboard.

3. Key State Machines

3.1 Student-Exam Mapping Status

```
CREATED → LOCKED → REVEALED → SCANNED → VERIFIED
                                      \→ WRONG_HALL
                                      \→ FLAGGED
   (no-show by exam end) → ABSENT
```

- `CREATED`: mapping exists, barcode token generated, hall/seat data withheld from all API responses.
- `LOCKED` : student can see the pass and barcode, but hall/seat fields remain withheld; countdown displayed.
- `REVEALED`: server-side reveal job has flipped the flag at T-30; hall/seat now included in API responses and pushed to the client.
- `SCANNED`: invigilator has scanned the barcode; awaiting verification decision.
- `VERIFIED` / `WRONG_HALL` / `FLAGGED`: terminal states written to `verification_logs`.
- `ABSENT`: system-assigned if no scan is recorded by a configurable cutoff after exam start (e.g., 30 minutes in), surfaced to the admin dashboard for follow-up.

3.2 Barcode Token Lifecycle

```
ISSUED (on mapping creation)
   → DISPLAYED (rotating short-lived render token, refreshed every 60-120s on the student portal)
   → SCANNED (invigilator reads a valid, unexpired token)
   → CONSUMED (server marks mapping used_at; further scans rejected as ALREADY_USED)
```

3.3 Invigilator Scan Sync State

```
SCANNED_LOCALLY (SQLite row written, sync_status = false)
   → QUEUED (awaiting network)
   → SYNCING (batch POST in flight)
   → SYNCED (server ack received, sync_status = true)
       \→ SYNC_CONFLICT (duplicate mapping already verified elsewhere → auto-resolved to earliest verified_at, later one FLAGGED)
```

4. Reveal Trigger Flow (Detailed)

1. A scheduled worker (Redis-backed job queue) continuously checks upcoming `exams.start_time` values.
2. For each exam, at `start_time - reveal_threshold`, the worker updates all associated `student_exam_mappings` to `reveal_unlocked = true`.
3. The worker emits a `reveal:{studentId}` event over WebSocket to any connected student portal session.
4. The student portal, on receiving the event (or on next poll if disconnected), re-fetches `/api/v1/student/exams/:examId/pass`, which now includes hall/seat fields, and animates the unlock.
5. If a student's session was never connected (e.g., app closed), the very next page load after T-30 simply returns the unlocked payload directly — no missed reveal state is possible since the source of truth is server-side, not a client timer.

5. Verification Flow (Detailed)

1. Invigilator's camera captures a barcode; the app decodes the signed token payload (`mapping_id`, `exam_id`, expiry, signature).
2. **Online mode**: token is POSTed to `/api/v1/invigilator/verify`; server validates signature, expiry, and `used_at` status, checks the mapping's `hall_id` against the invigilator's assigned hall, and returns a decision (`VERIFIED` / `WRONG_HALL` / `ALREADY_USED`) plus the candidate profile.
3. **Offline mode**: the same validation logic runs against the locally cached roster and public verification key; the decision is rendered instantly and queued for later server sync.
4. The app renders the appropriate visual state (green/red/amber) and awaits or auto-applies the invigilator's confirmation.
5. On confirmation, a `verification_logs` row is created (locally first, then synced) with `mapping_id`, `invigilator_id`, `verified_at`, and `status`.
6. The Live Attendance Dashboard (admin side) receives the update via WebSocket (online path) or on the next batch sync (offline path) and increments the relevant hall counter.